

Relatório do Trabalho Prático 1 Programação Paralela com OpenMP

Aron P. Sarabia, Cassiano C. De Souza

Faculdade de Computação (FACOM)
Universidade Federal do Mato Grosso do Sul (UFMS) – 79070-900
A Av. Costa e Silva, s/nº – Bairro Universitário
Campo Grande – MS – Brasil

{aron_petta, cassiano_c_souza}@ufms.br

Resumo. O presente trabalho foca em apresentar uma resolução ao Método Iterativo de Jacobi através de programação paralela via OpenMP. Através de análise de desempenho, verificaremos os principais pontos de reflexão sobre os limites e desafios encontrados na paralelização do algoritmo do método.

1. Introdução

O Método de Jacobi resolve um Sistema Linear através de sucessivas iterações de aproximação à um vetor-solução escolhido previamente. Quando há convergência com a solução, o sistema é interrompido.

A implementação do paralelismo no algoritmo do método inclui fragmentar os processos do laço de iteração e relacionar os mesmos a threads específicas para otimizar a resolução. Isso permite que diferentes cálculos sejam processados em paralelo, evitando atrasos e dependências que poderiam comprometer o desempenho do algoritmo. Em contraponto, a técnica promove maior velocidade e precisão na busca pela solução do problema.

2. Implementação

Em primeira instância, antes da compilação do código, o usuário define o tamanho da matriz de vetores-solução.

Em seguida, a Main chama um conjunto de funções-teste para buscar os resultados do Método de Jacobi, mediante o tamanho pré definido de iterações e tolerância para convergência. Todos os chamados são iniciados e finalizados com uma função que calcula o tempo de execução das mesmas. Dentro desses chamados, uma função gera uma matriz com vetores-solução, outra realiza a implementação sequencial da mesma, uma terceira realiza a implementação paralela e uma última imprime os resultados dessas

implementações, estruturando a saída em formato de tabela com todos os dados de comparação de desempenho.

A função de implementação sequencial inicia gerando uma nova lista de vetores-solução; logo mais, garante que o loop do laço comece e que o erro permaneça acima da tolerância; então chama uma estrutura para guardar os dados da iteração e do erro; e continua as iterações até convergir ou atingir o limite. Durante as iterações, percorre cada linha e coluna através de dois laços (*for*), para no fim realizar as somas de valores. No fim, calcula também o erro.

Já na função paralela, usa-se o ***#pragma omp parallel for*** para paralelizar a função com laços do tipo ***for***. Nesse laço, as iterações de linha da matriz são partilhadas com as threads, mediante o número de threads disponíveis, usando o ***schedule(static)***. Dessa forma, quando a iteração ocorre, cada thread disponível e alocada trabalha com uma parte do problema, realizando diversas tarefas ao mesmo tempo. Em virtude desse comportamento, também utiliza-se o ***reduction(+=error)*** para garantir que os dados obtidos nessas iterações sejam corretamente atualizados sem comprometer a integridade das respostas, enquanto o ***barrier***, naturalmente já implementado nos laços, garante que as tarefas sejam corretamente processadas e o programa não continue antes de terminarem suas execuções.

Ao final de toda a execução, os dados são estruturados em uma tabela, relacionados de forma comparativa (estrutural X paralelizado) e printados para o usuário.

3. Ambiente Experimental

- Processador (modelo e frequência): 11th Gen Intel® Core™ i5-1135G7.
- Número de núcleos e threads: 4 Núcleos e 8 Threads.
- Memória RAM: 24,0 GiB
- Sistema operacional: Fedora Linux 43 (Workstation Edition)
- Compilador e versão: g++ (GCC) 15.2.1 20260123 (Red Hat 15.2.1-7)

4. Resultados

4.1. Tempos de execução

Abaixo foram ilustrados em gráficos, separados pelo tamanho, os dados das seguintes matrizes utilizadas: Matriz de 250, Matriz de 500, Matriz de 1000 e Matriz de 1500.

Gráfico do Tempo (Número de Threads X Tempo em Segundos)

Grafico Tempo(s)

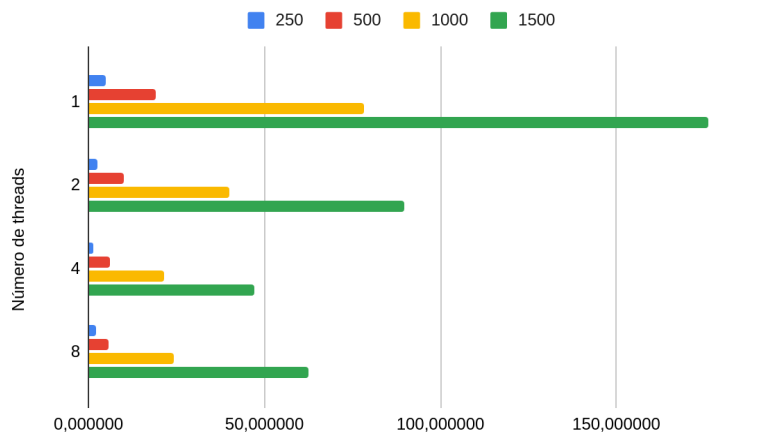


Gráfico do Speedup (Número de Threads X Potência)

Grafico do Speedup(x)

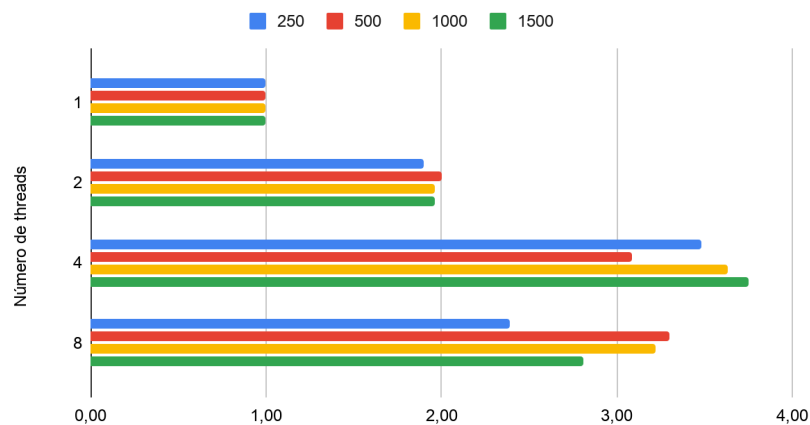
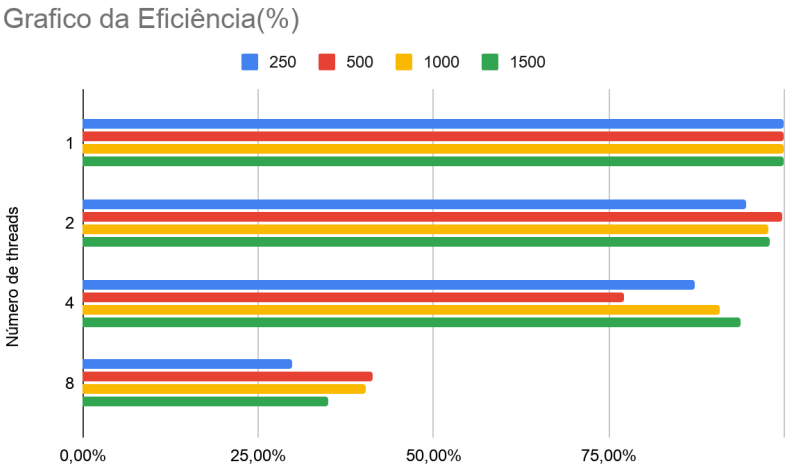


Gráfico da Eficiência(Número de Threads X Porcentagem de uso das threads)



4.2. Convergência

Tabela de convergência do método de Jacobi			
Tamanho da Matriz	Iterações até convergir	Erro final	Tolerância
250	20000	0,024444	1,00E-06
500	20000	0,428079	1,00E-06
1000	20000	0,799085	1,00E-06
1500	20000	0,911488	1,00E-06

4.3. Discussão

- O **impacto no número de threads** dependerá essencialmente de qual é o processador, visto que há diversos processos paralelos que cuidam de criar e destruir processos paralelizados, e isso é extremamente custoso. Nesse sentido, o melhor número de threads para esse problema é 4, pois foi o que demonstrou melhor desempenho.
- A **limitação lógica** foi a de tentar entender o por que um número maior de threads (superior à 4) estava levando mais tempo; e também descobrir qual era a melhor quantidade de interações para tentar ser o mais preciso possível.
- Como pode ser visto, a **paralelização tem maior destaque com matrizes maiores** pois utilizamos “`schedule(static)`”, que divide o tamanho do problema para as N threads, dividindo assim a carga de forma mais balanceada: e quanto maior o problema melhor fica a distribuição para cada thread.

5. Conclusão

A solução paralela, implementada com o algoritmo do Método de Jacobi, supera os problemas da execução sequencial ao distribuir o processamento entre múltiplas threads, tornando a execução mais rápida e dinâmica. Dessa forma, os resultados são obtidos em um tempo consideravelmente menor quando comparados ao sequencial, com um ganho de desempenho evidente. Além disso, também é possível observar crescimento gradual de eficiência conforme os valores aumentam, evidenciando que quanto maior a matriz utilizada nesse método, melhor o desempenho do algoritmo.

Assim, a solução paralela apresenta desempenho geral superior, mantendo tempos reduzidos mesmo em entradas maiores e demonstrando escalabilidade mais consistente. Comparada ao método sequencial, ela se mostra a abordagem mais adequada para aplicações que lidam com grandes volumes de dados.